

# 1. Abstraction

# 2. Abstraction

# 3. Abstraction

*Haskore music notation*  
– *An algebra of music* –

PAUL HUDAK, TOM MAKUCEVICH, SYAM GADDE  
AND BO WHONG

*Department of Computer Science, Yale University,  
New Haven, CT 06520, USA  
(e-mail: [hudak@cs.yale.edu](mailto:hudak@cs.yale.edu))*



# Music Notation

1

## Fuga a 3 voci

JOHANN SEBASTIAN BACH (1685-1750)

BWV 847

The image displays three systems of musical notation for Johann Sebastian Bach's Fuga a 3 voci, BWV 847. Each system consists of a treble staff and a bass staff. The first system shows the beginning of the piece with a treble staff starting on a G4 and a bass staff starting on a G3. The second system shows the continuation of the melody in the treble staff. The third system shows the continuation of the melody in the treble staff and the beginning of the bass staff melody. The notation includes various musical symbols such as notes, rests, and accidentals.

# MIDI

A MIDI file always starts with a header chunk, and is followed by one or more track chunks.

MThd <length of header data>

<header data>

MTrk <length of track data>

<track data>

MTrk <length of track data>

<track data>

...

<Header Chunk> = <chunk type> <length> <format> <ntrks> <division>

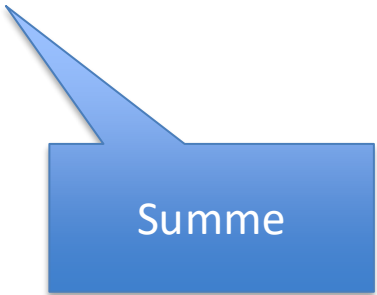
<Track Chunk> = <chunk type> <length> <MTrk event>+

<MTrk event> = <delta-time> <event>

<event> = <MIDI event> | <sysex event> | <meta-event>



Produkt



Summe

# Problems

- no representation for composer intention
- specific to human players
- specific to western, tonal music
- (no notation for algorithmic music)

# Generalizing Music Notation



(a)

numerator : denominator



(b)



(c)



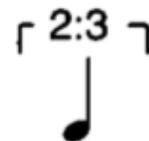
(d)



(e)



(f)



(g)

# Haskore

```
data Note = Note Pitch Dur | Rest Dur
    deriving Text
```

```
type Line = [Note]
```

```
data Music
    = Line Line
    | Scale Int Int Music
    | Trans Int Music
    | Music :+: Music
    | Music :=: Music
    | Instr IName Music
    deriving Text
```

```
type IName = String
```

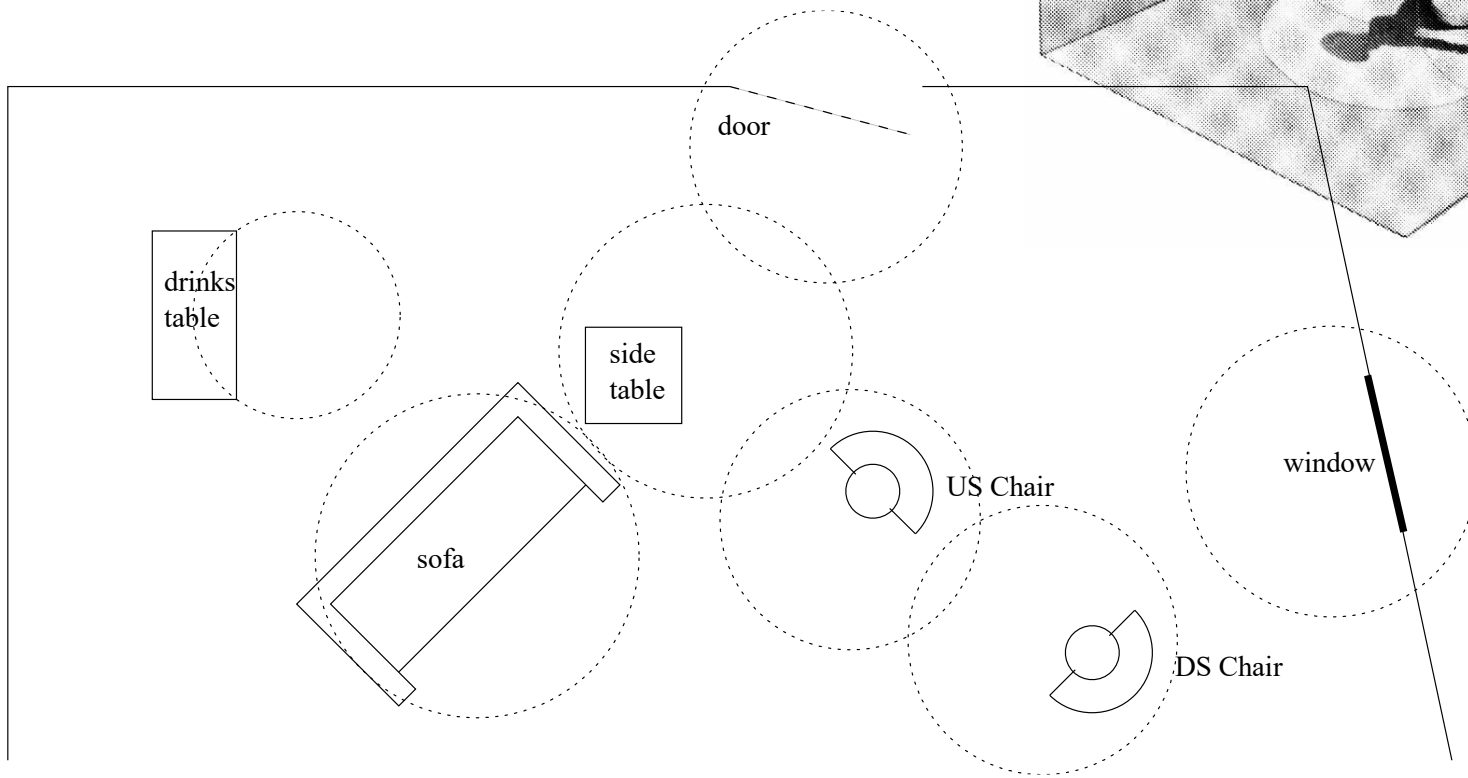
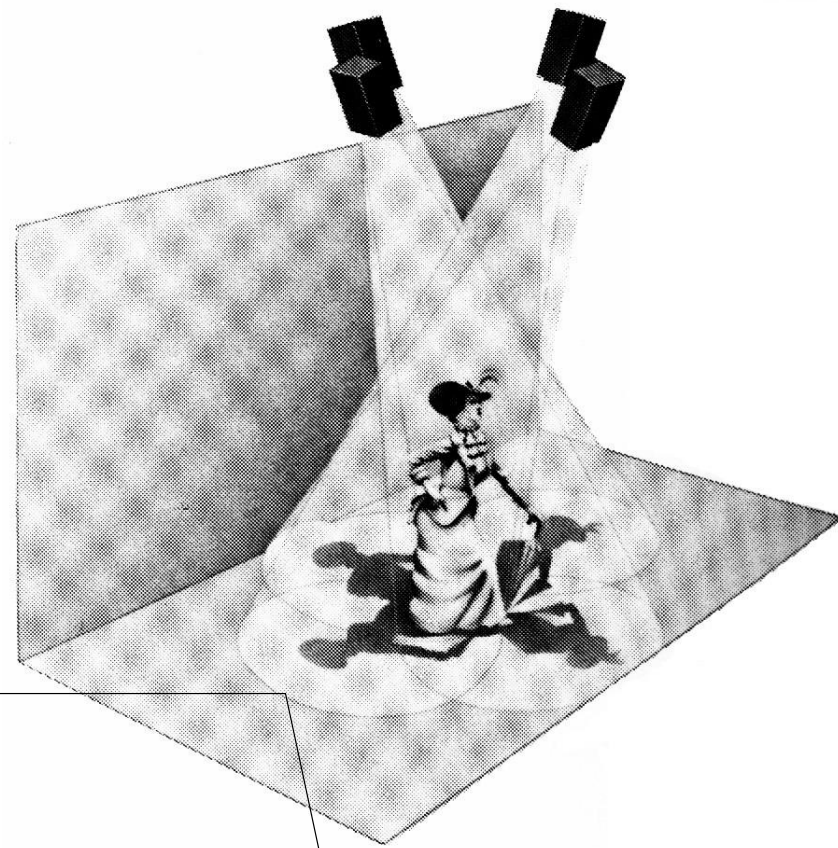
# Data Modeling for Music

- much smaller than MIDI
- defined semantics
- domain understanding as equations
- satisfies future requirements

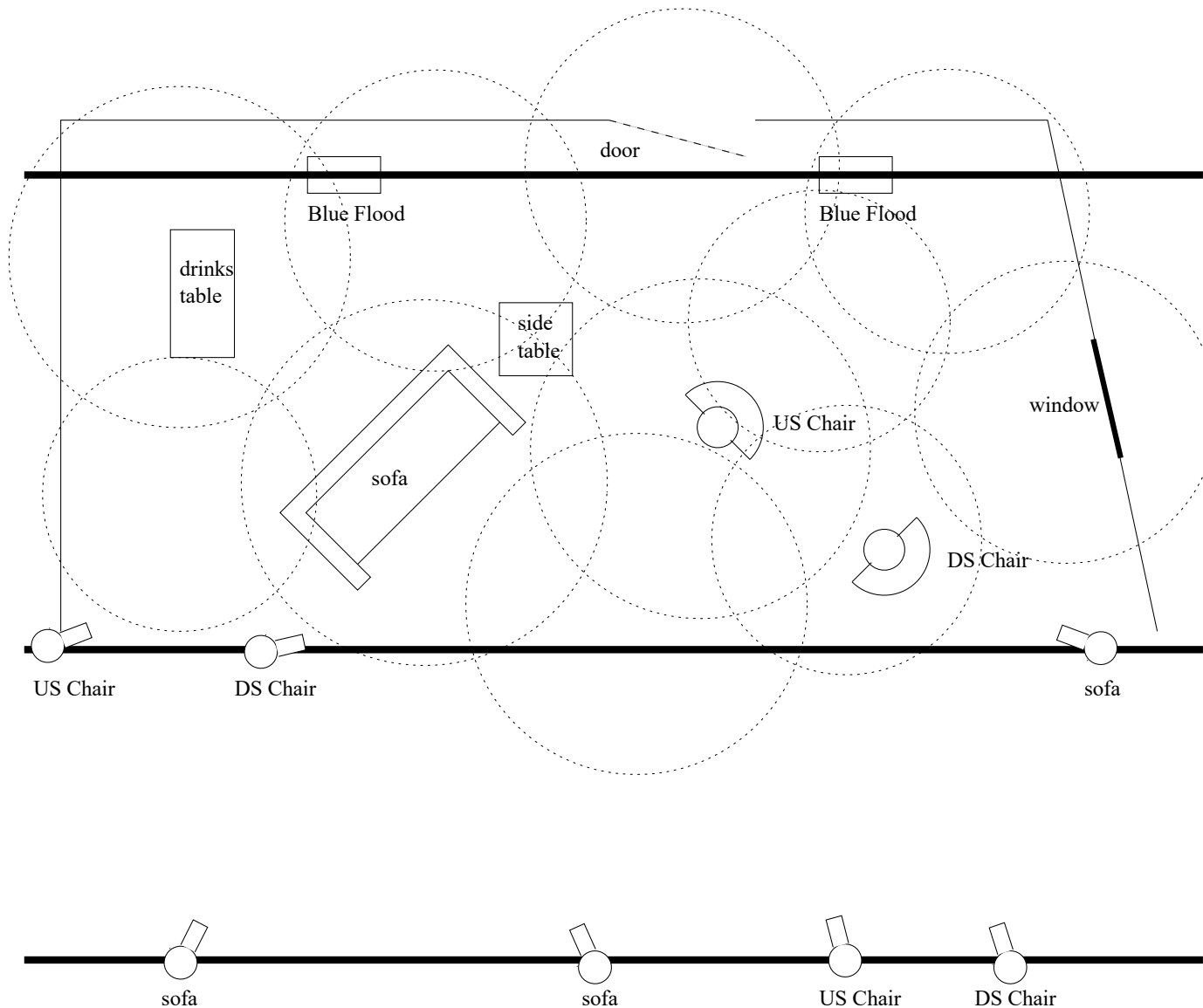
# Stage Lighting



# Lighting Design



# Specification vs. Implementation



# Data Modeling for Stage Lighting

```
type Fixture = Int
```

```
type Factor = Double
```

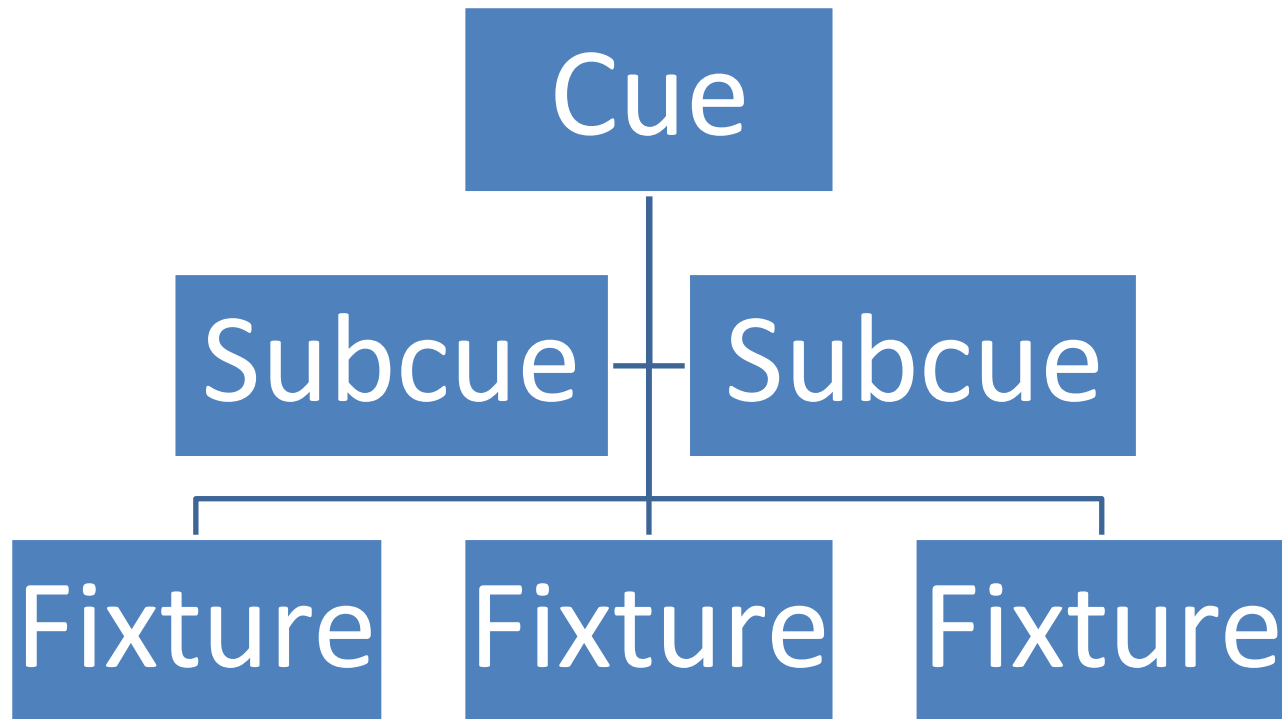
```
type Cue = Map Fixture Factor
```

# Lighting Control Systems

- “Submaster”
- “relative Cues”
- “Presets”
- ...



# Stratified Model

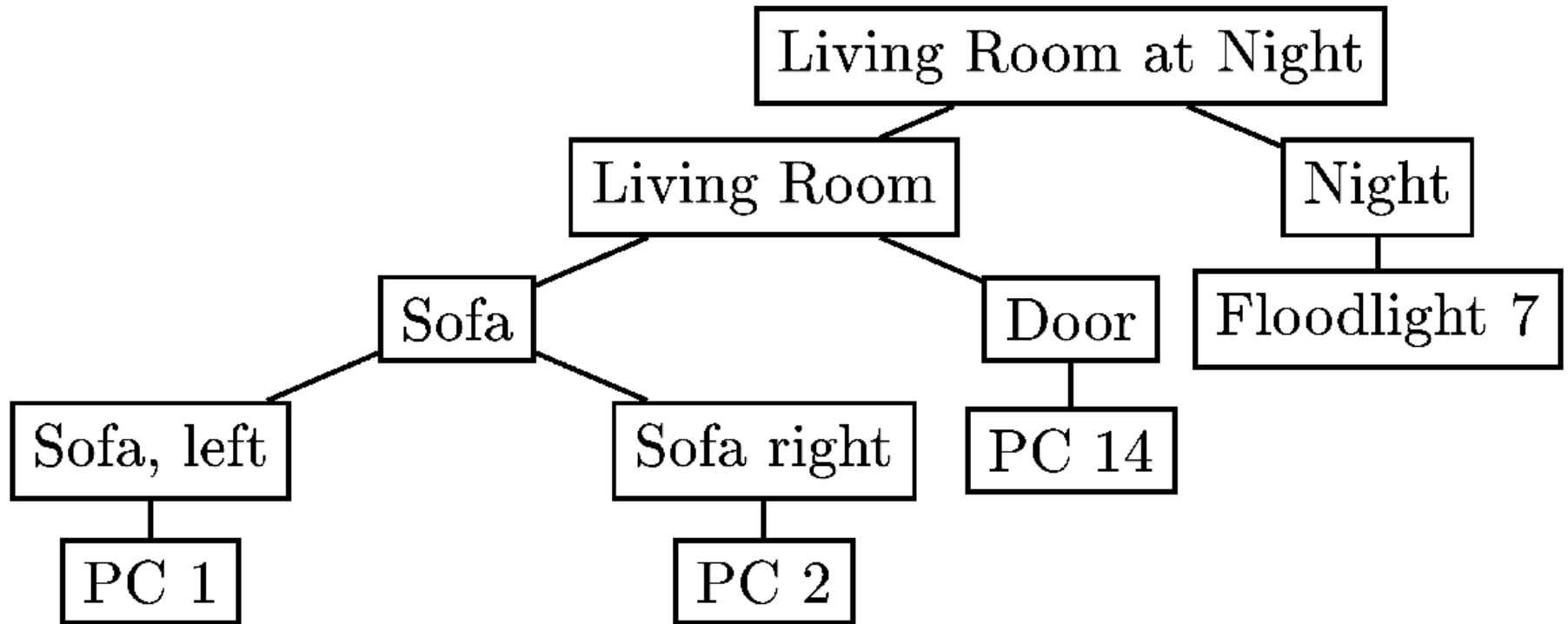


# Stratified Model

```
type Cue = [SubCue]
```

```
type SubCue = Map Fixture Factor
```

# Hierarchical Structure



# Combinator Model

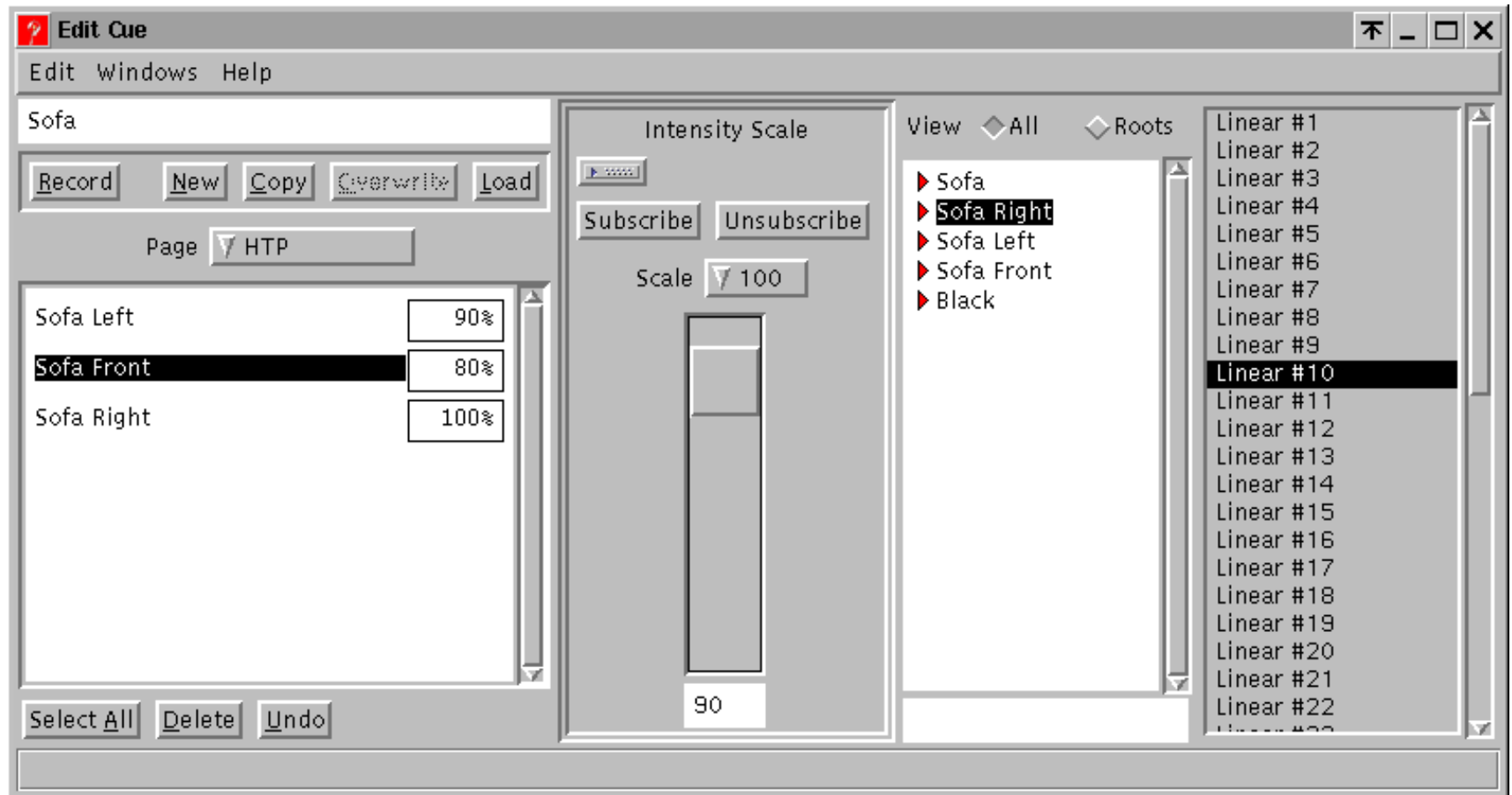
Abstraction!

A **Cue** is ...

- ... a **single** lamp
- ... a **scaled** cue
- ... a **combination** of cues

```
data Cue
  = Single Fixture
  | Scaled Factor Cue
  | Combination [Cue]
```

# Lula



# Advantages

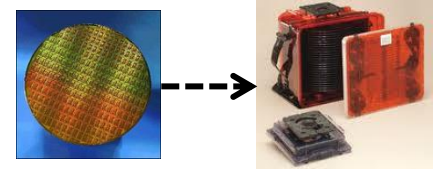
- **much** easier to use
- **much** faster to use
- does not need a big console
- tracks changes
- works on tour
- long-lived



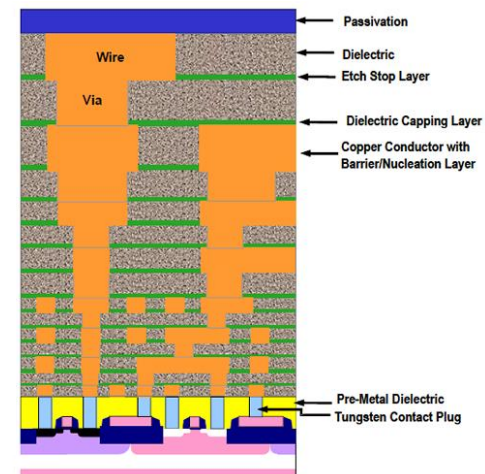


# Semiconductor Manufacturing Process

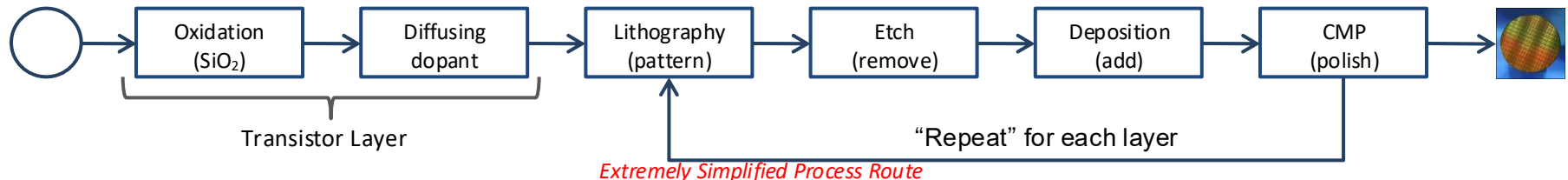
- The product Wafer contains many chips (devices)
  - Wafers are contained in FOUPs for movement through the factory



- The chip has multiple layers to create the functioning device
  - The first few layers are the transistors
  - The remaining layers are the interconnection “wiring”



- The manufacturing process moves the wafers through multiple equipment to fabricate each layer



- Each step in the route has a recipe that is unique for the given product.

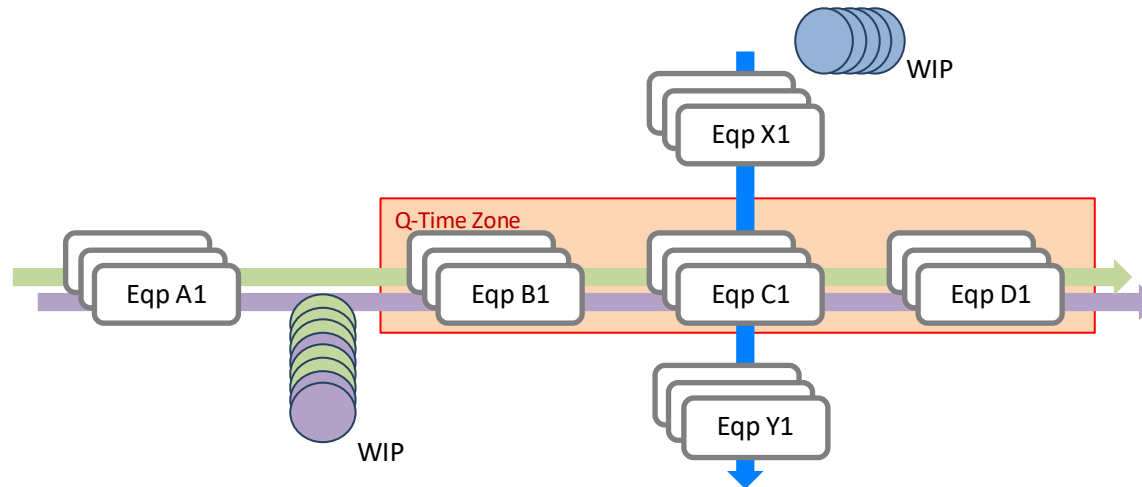
# Route Representation

```
data Operation = TrackIn | Process | TrackOut
```

```
type Route = [Operation]
```

```
r1 = [TrackIn, Process, Process, TrackOut]
```

# Q-Time Zones



Per Route:

- 1000 Operations
- 50 separate Q-Time Zones

# Q-Time Zones

```
data Operation = TrackIn | Process | TrackOut
```

```
type Route = [RouteElement]
```

```
data RouteElement =  
    RouteOp Operation  
    | RouteQTZone Duration [Operation]
```

# Q-Time Zones

```
data Operation = TrackIn | Process | TrackOut
```

```
type Route = [RouteElement]
```

```
data RouteElement =  
    RouteOp Operation  
    | RouteQTZone Duration [Operation]
```

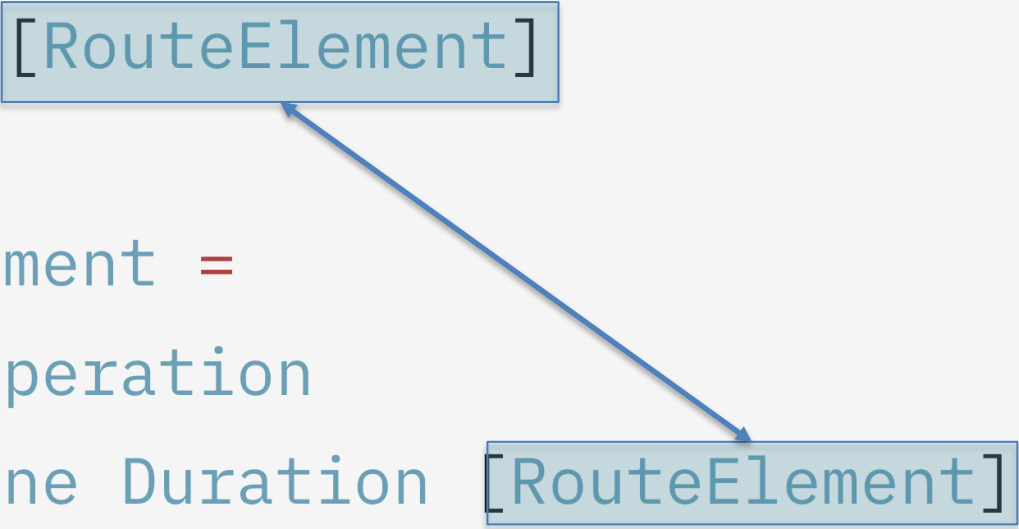


# Q-Time Zones

```
data Operation = TrackIn | Process | TrackOut
```

```
type Route = [RouteElement]
```

```
data RouteElement =  
    RouteOp Operation  
    | RouteQTZone Duration [RouteElement]
```



# Q-Time Zones

```
data Operation = TrackIn | Process | TrackOut
```

```
type Route = [RouteElement]
```

```
data RouteElement =  
    RouteOp Operation  
    | RouteQTZone Duration
```

Route



# Nested Q-Time Zones

```
r3 = [RouteOp TrackIn,  
      RouteQTZone 5  
      [RouteOp Process,  
        RouteQTZone 7  
        [RouteOp Process, RouteOp Process]]]
```